



Mekelle University
Mekelle Institute of Technology
Department of Information Technology

Advanced Database Systems

2011 E.C, Sem: II
Years: II&III

By: Mezgebe Mehari

mezgebem@gmail.com

0914 21 12 72



Chapter – One

Query Processing and Optimization

1. *Overview of Query Optimization*
2. *Translating SQL Queries into Relational Algebra*
3. *Basic Algorithms for Executing Query Operations*
4. *Using Heuristics in Query Optimization*
5. *Using Selectivity and Cost Estimates in Query Optimization*
6. *Overview of Query Optimization in ORACLE*
7. *Semantic Query Optimization*



1.1) Overview of Query Optimization

What is Query Optimization?

- A **query** expressed in a **high-level query language** such as SQL must first be **scanned**, **parsed**, and **validated**.
- The **scanner** identifies the language **tokens** such as:
 - ❖ *SQL keywords.*
 - ❖ *Attribute names.* and
 - ❖ *Relation names.*
- Whereas the **parser checks** the *query syntax* to determine whether it is *formulated according to the syntax rules* (rules of *grammar*) of the query language.
- The query must also be **validated**, by *checking* that all **attribute** and *relation names* are **valid** and *semantically meaningful names* in the schema of the particular database being queried.



1.1) Overview of Query Optimization

What is Query Optimization?

- ❑ An *internal representation* of the query is then created, usually *as a tree data structure* called a **query tree**.
 - ✓ It is also possible to represent the query using a *graph data structure* called a **query graph**.
- ❑ The **DBMS** must then devise an **execution strategy** for *retrieving* the result of the query from the database files.
 - ✓ A query typically has many *possible execution strategies*, and the *process of choosing* a **suitable** one for *processing* a **query** is known as **query optimization**.
- ❑ The term **optimization** is actually a *misnomer* because in some cases the chosen *execution plan* is not the *optimal (best) strategy*—it is just a reasonably *efficient strategy* for executing the query.
 - ✓ Hence, **planning** of an *execution strategy* may be a more accurate description than **query optimization**.



1.2) Translating SQL Queries into Relational Algebra

- ✓ In **practice**, SQL is the query language that is used in most **commercial RDBMSs**.
 - ❖ An **SQL query** is *first translated* into an equivalent extended **relational algebra expression**—represented as a query **tree data structure**—that is then **optimized**.
- ✓ **Typically**, SQL queries are *decomposed* into **query blocks**, which form the *basic units* that can be *translated* into the *algebraic operators* and **optimized**.
 - ❖ A query **block** contains a single **SELECT-FROM-WHERE** expression, as well as **GROUP BY** and **HAVING** clauses (if they're part of the block).
- ✓ Hence, *nested queries* within a **query** are identified as *separate query* blocks.
 - ✓ Because **SQL includes aggregate operators**—such as **MAX, MIN, SUM, and COUNT**—these operators must also be included in the extended algebra, as we discussed in Fundamentals of DB Sys.



1.2) Translating SQL Queries into Relational Algebra

- ✓ Consider the following SQL query on EMPLOYEE Relation:

```
SELECT LNAME, FNAME
FROM   EMPLOYEE
WHERE  SALARY >      (SELECT   MAX (SALARY)
                      FROM     EMPLOYEE
                      WHERE     DNO=5);
```

- ✓ This query includes a *nested subquery* and hence would be **decomposed** into *two blocks* hereby:

Inner Block

```
(SELECT MAX
(SALARY) FROM
EMPLOYEE
WHERE  DNO=5)
```

Outer Block

```
SELECT LNAME,
FNAME FROM
EMPLOYEE WHERE
SALARY > C
```

- ✓ Where **C** represents the *result returned* from the *inner block*.



1.2) Translating SQL Queries into Relational Algebra

- ✓ The **inner block** could be **translated** into the *extended relational algebra* expression.

(SELECT MAX
(SALARY) FROM
EMPLOYEE
WHERE DNO=5)

To

MAX SALARY ($S_{DNO=5}$ (EMPLOYEE))

- ✓ And the **outer block** into the **expression**.

SELECT LNAME,
FNAME FROM
EMPLOYEE WHERE
SALARY > C

To

$p_{LNAME, FNAME} (S_{SALARY > C} (EMPLOYEE))$



1.2) Translating SQL Queries into Relational Algebra

- ✓ The **query optimizer** would then **choose** an **execution plan** for *each block*.
- ✓ We *should note that* in the above Eg, **the inner block** needs to be *evaluated only once* to produce the **maximum salary**, which is **then** used — as the **constant C** — by *the outer block*.
- ✓ We normally call such kind of nested query as an *uncorrelated nested query*.
- ✓ It is much **harder** to **optimize** the more complex **uncorrelated nested queries**, where a tuple variable from the **outer** block appears in the *WHERE-clause* of the **inner** block.



1. 3) Basic Algorithms for Executing Query Operations

I) External Sorting:

- ✓ **Sorting** is one of the *primary algorithms* used in query processing.
 - ✓ For *example*, whenever an **SQL** query specifies an **ORDER BY-clause**, the query result must be *sorted*.
- ✓ **External sorting** refers to sorting *algorithms that are suitable* for **large files** of records stored on **disk** that do not fit entirely in *main memory*, such as most database files.
 - ✓ The typical external sorting algorithm uses a **sort-merge strategy**, which starts by **sorting** small *sub-files – called runs* – of the main file and then merges the **sorted runs**, creating larger sorted *sub-files* that are merged in turn.
- ✓ The *sort-merge algorithm*, like other database algorithms, requires *buffer space* in main memory, where the actual **sorting** and **merging** of the **runs** is performed.



1.3) Basic Algorithms for Executing Query Operations

External Sorting:

✓ It consists of two phases:

- (1) The **sorting** phase and
- (2) The **merging** phase.

- ✓ In the **sorting phase**, runs (**portions** or **pieces**) of the file that can fit in the available buffer space are *read into main memory*, sorted using an *internal sorting algorithm*, and written back to disk as temporary sorted **sub-files** (or runs).
- ✓ The **size** of a run and *number of initial runs* is dictated by the *number of file blocks* and the *available buffer space*.
- ✓ For example, if $b = 5$ blocks and the size of the file $B = 1024$ blocks, then $n = \lceil \frac{B}{b} \rceil$, or 205 initial runs each of size 5 blocks (except the last run which will have 4 blocks). Hence, after the sort phase, 205 sorted runs are stored as temporary **sub-files** on disk.



1.3) Basic Algorithms for Executing Query Operations

External Sorting:

- ✓ In the **merging phase**, the sorted **runs** are merged during one or *more passes*.
- ✓ The **degree of merging** is the *number of runs* that can be merged **together** in each pass.
 - ✓ In **each** pass, one **buffer** block is needed to *hold one block* from each of the runs *being merged*, and one block is needed for containing *one block* of the merge result.
- ✓ The *first term* represents the *number of block accesses* for the **sort** phase, since each file block is *accessed twice*—once for reading *into memory* and once for writing the records back to disk after sorting.
- ✓ The **second term** represents the *number of block* accesses for the merge phase, *assuming* the worst-case.



1.3) Basic Algorithms for Executing Query Operations

II) Implementing the SELECT Operation:

- ✓ There are many options for **executing** a **SELECT** operation; some depend on the *file having specific access paths* and *may apply only to certain types* of selection conditions.
- ✓ We will use the following operations, specified on the relational database, to *illustrate* our discussion:

(OP1): **S**_{SSN='123456789'}(EMPLOYEE)

(OP2): **S**_{DNUMBER>5}(DEPARTMENT)

(OP3): **S**_{DNO=5}(EMPLOYEE)

(OP4): **S**_{DNO=5 AND SALARY>30000 AND SEX='F'}(EMPLOYEE)

(OP5): **S**_{ESSN='123456789' AND PNO=10}(WORKS_ON)



1.3) Basic Algorithms for Executing Query Operations

Search Methods for Simple Selection:

- ✓ A number of **search algorithms** are possible for *selecting records from a file*.
 - ✓ These are *also known as file scans*, because they *scan the records of a file* to search for and retrieve records that satisfy a *selection condition*.
- ✓ If the search *algorithm involves the use of an index*, the index search is *called an index scan*.
 - ✓ The following *search methods (S1 through S6)* are examples of some of the *search algorithms* that can be used to implement a **select** operation:



1.3) Basic Algorithms for Executing Query Operations

Search Methods for Simple Selection:

- ❖ **S1. Linear search (brute force):** Retrieve *every record* in the file, and test whether its *attribute values satisfy* the *selection* condition.
- ❖ **S2. Binary search:** If the *selection* condition involves an *equality comparison* on a *key attribute* on which the file is ordered, *binary search*—which is more **efficient** than *linear search*—can be used. An example is **OP1** if **SSN** is the ordering attribute for the EMPLOYEE file.
- ❖ **S3. Using a primary index (or hash key):** If the *selection* condition involves an equality comparison on a *key attribute* with a *primary index* (or *hash key*)—for example, $_{SSN} = '123456789'$ in OP1—use the *primary index* (or hash key) to retrieve the record. Note that this condition retrieves a *single record* (at most).



1.3) Basic Algorithms for Executing Query Operations

- ❖ *S4. Using a primary index to retrieve multiple records:* If the *comparison condition* is $>$, $\geq$, $<$, or $\leq$ on a **key field** with a *primary index* —
 - ❖ for example, $\text{DNUMBER} > 5$ in OP2—use the index to *find the record satisfying the corresponding equality condition* ($\text{DNUMBER} = 5$), then retrieve all subsequent records in the (ordered) file. For the *condition* $\text{DNUMBER} < 5$, retrieve all the preceding records.
- ❖ *S5. Using a clustering index to retrieve multiple records:* If the *selection condition* involves *an equality comparison on a non-key attribute* with a clustering index —
 - ❖ for example, $\text{DNO} = 5$ in OP3—use the index to retrieve all the records satisfying the *condition*.
- ❖ *Using a secondary index on an equality comparison:* This search method can be used to *retrieve a single record if the indexing field is a key* (has unique values) or *to retrieve multiple records if the indexing field is not a key*. It also uses $>$, $\geq$, $<$, or $\leq$



1.3) Basic Algorithms for Executing Query Operations

III) Implementing the JOIN Operation:

- ✓ The *JOIN* operation is one of the *most time-consuming* operations in *query* processing.
- ✓ Many of the join operations encountered in queries are of the *EQUIJOIN* and *NATURAL JOIN* varieties, so we consider only these *two* here.
 - ✓ For the remainder of this chapter, the term *join* refers to an *EQUIJOIN* (or *NATURAL JOIN*).
- ✓ There are many possible ways to implement a *two-way join*, which is a join on *two files*.
- ✓ The *algorithms* we consider are for *join* operations of the form:

$$R_{A=B} S$$

- ✓ Where *A* and *B* are *domain-compatible attributes* of *R* and *S*, *respectively*.



1.3) Basic Algorithms for Executing Query Operations

- ✓ We illustrate **four** of the *most common techniques* for performing join, using the *following example* operations:
 - ✓ (OP6): EMPLOYEE_{DNO=DNUMBER} DEPARTMENT
 - ✓ (OP7): DEPARTMENT_{MGRSSN=SSN} EMPLOYEE

Methods for Implementing Joins:

- ✓ *J1. Nested-loop join (brute force):*
 - For each *record t in R* (outer loop), retrieve every *record s from S* (inner loop) and test whether the *two records satisfy* the join condition $t[A] = s[B]$.
- ✓ *J2. Single-loop join (using an access structure to retrieve the matching records):*
 - If an index (or *hash key*) *exists* for one of the *two join attributes* — say, B of S — retrieve each record t in R, one at a time (single loop), and then use the *access structure to retrieve directly* all matching records s from S that satisfy $s[B] = t[A]$.



1.3) Basic Algorithms for Executing Query Operations

Methods for Implementing Joins:

✓ J3. Sort-merge join:

- If the records of **R** & **S** are *physically sorted* by value of the join attributes **A** and **B**, respectively, we can implement *the join in the most efficient* way possible.

✓ J4. Hash-join:

- The records of files **R** and **S** are both **hashed** to the *same hash file*, using the *same hashing* function on the join attributes *A of R* and *B of S* as **hash keys**.
- **First**, a *single pass* through the file with *fewer records* (say, R) hashes its records to the *hash file buckets*; this is called the **partitioning phase**, since the records of R are *partitioned into the hash* buckets.
- In the **second phase**, called the **probing phase**, a single pass through the other file (S) then **hashes** each of its records to *probe* the appropriate bucket, and that record is combined with all *matching records from R* in that **bucket**.



1.4) Using Heuristics in Query Optimization

- **Optimization techniques** that apply **heuristic rules**, modify the *internal representation* of a query—which is **usually** in the form of a **query tree** or a **query graph** data structure—to improve its expected **performance**.
- The **parser** of a *high-level query* **first** generates an *initial internal representation*, which is then *optimized* according to *heuristic rules*.
 - Following that, a *query execution plan* is generated to execute groups of operations *based on the access paths* available on the **files** involved in the query.
- One of the main **heuristic rules** is to apply **SELECT** and **PROJECT** operations *before* applying the **JOIN** or other binary operations.
 - Because the size of the **file resulting** from a **binary** operation like JOIN is usually a *multiplicative function* of the sizes of the input files.
 - The **SELECT** and **PROJECT** operations reduce the size of a file and hence should be applied *before* a join or *other binary* operation.



1.4) Using Heuristics in Query Optimization

Notation for Query Trees and Query Graphs:

- ❑ A **query tree** is a *tree data structure* that corresponds to a *relational algebra* expression.
 - ❑ It represents the **input relations** of the query as *leaf nodes* of the tree, and represents the *relational algebra operations* as **internal** nodes.
 - ❑ An execution of the query *tree consists of executing* an *internal node operation* whenever its operands are available and then *replacing* that *internal node* by the relation *that results from executing* the operation.
- ❑ The execution *terminates* when the *root node is executed* and produces the result relation for the query.
- ❑ Example: $\text{DNUM}=\text{DNUMBER}(\text{DEPARTMENT})\text{MGRSSN}=\text{SSN}(\text{EMPLOYEE})$ Corresponds to

this query: **SELECT** P.PNUMBER, P.DNUM, E.LNAME, E.ADDRESS, BDATE
FROM PROJECT **AS** P, DEPARTMENT **AS** D, EMPLOYEE **AS** E
WHERE P.DNUM=D.DNUMBER **AND** D.MGRSSN=E.SSN **AND**
P.LOCATION='MIT';



1.4) Using Heuristics in Query Optimization

- ❑ The three relations **PROJECT**, **DEPARTMENT**, and **EMPLOYEE** are represented by *leaf nodes P, D, and E*, while the *relational algebra operations* of the expression are *represented by internal tree nodes*.
- ❑ As we can see, the *query tree represents* a **specific** order of operations for *executing a query*.
- ❑ A more *neutral representation* of a query is the **query graph** notation.
 - ❑ **Relations** in the query are represented by **relation nodes**, which are displayed **as single circles**. *Constant values*, typically from the query selection conditions, are represented by **constant nodes**, which are displayed as *double circles*.
 - ❑ **Finally**, the **attributes** to be retrieved from **each relation** are displayed in **square brackets** above each relation.
- ❑ The **query graph** *does not indicate an order* on which **operations** to *perform first*. There is *only a single graph* corresponding to **each query**.



1.5) Using Selectivity and Cost Estimates in Query Optimization

- ❑ A query optimizer should *not depend solely on heuristic rules*; it should also *estimate and compare the costs of executing a query using different execution strategies* and *should choose the strategy with the lowest cost estimate*.
- ❑ Hence, this approach is more suitable for **compiled queries** where the *optimization is done at compile time* and *the resulting execution strategy code* is stored and executed directly at runtime. For **interpreted queries**, where the *entire process occurs at runtime*.
 - ❑ We call this approach *cost-based query optimization*, and it uses *traditional optimization techniques* that search the *solution space* to a problem for a solution that *minimizes an objective (cost)* function.
- ❑ The *cost functions* used in query optimization are *estimates and not exact cost functions*, so the optimization may *select a query execution strategy* that is *not the optimal* one.



1.5) Using Selectivity and Cost Estimates in Query Optimization

Cost Components for Query Execution:

❑ The cost of *executing a query* includes the following components.

1) Access cost to secondary storage:

- This is the *cost of searching* for, **reading**, and **writing** data blocks that *reside on secondary storage*, mainly on **disk**.
- The cost of *searching for records* in a file depends on the *type of access structures* on that file, such as *ordering*, *hashing*, and *primary or secondary* indexes.
- In addition, **factors** such as whether the **file blocks** are allocated **contiguously** on the same disk **cylinder** or scattered on the disk affect the **access** cost.

2) Storage cost:

- This is the *cost of storing* any *intermediate files* that are generated by *an execution strategy* for the query.



1.5) Using Selectivity and Cost Estimates in Query Optimization

Cost Components for Query Execution:

3) Computation cost :

- This is the **cost** of performing **in-memory operations** on the data *buffers during query execution*.
- Such operations include **searching** for and **sorting records**, **merging records** for a join, and **performing computations** on field values.

4) Memory usage cost:

- This is the *cost pertaining* to the *number of memory buffers needed* during *query execution*.

5) Communication cost:

- This is the *cost of shipping the query* and *its results from the database site* to the **site or terminal** where the *query originated*.



1.5) Using Selectivity and Cost Estimates in Query Optimization

Catalog Information Used in Cost Functions:

❑ To *estimate* the *costs* of *various execution strategies*, we must keep track of *any information* that is needed for the *cost functions*.

• This *information* may be stored in the *DBMS catalog*, where it *is accessed* by the *query optimizer*.

• **First**, we must know the *size of each file*.

- For a file whose records are all of the *same type*, the *number of records (tuples) (r)*, the (average) *record size (R)*, and the *number of blocks (b)* (or close estimates of them) are needed.
- The *blocking factor (bfr)* for the file may also be needed.

❑ We must *also keep track* of the *primary access method* and the *primary access attributes* for each file.

- The *file records* may be *unordered*, *ordered* by an attribute with or without a *primary* or *clustering* index, or *hashed* on a *key* attribute.



1.5) Using Selectivity and Cost Estimates in Query Optimization

Catalog Information Used in Cost Functions:

- Information is kept on *all secondary indexes* and *indexing attributes*.
 - The *number of levels (x)* of each *multilevel index* (*primary, secondary, or clustering*) is needed for *cost functions* that estimate the number of *block accesses* that occur *during query* execution. In some *cost* functions the *number of first-level index blocks* is needed.
- Another important parameter is the *number of distinct values (d)* of an attribute and its *selectivity (sl)*, which is the *fraction* of records satisfying an *equality condition* on the attribute.
- This allows estimation of the *selection cardinality* ($s = sl * r$) of an attribute, which is the *average* number of records that will *satisfy an equality selection condition* on that attribute.



1.5) Using Selectivity and Cost Estimates in Query Optimization

Catalog Information Used in Cost Functions:

- For a *key attribute*, $d = r$, $sl = 1/r$ and hence $s = sl * r$, since $sl = 1/r$ then $s = 1/r * r = 1$.
- For a *non-key attribute*, by making an assumption that the d distinct values are *uniformly distributed* among the records, we *estimate* $sl = (1/d)$ and hence $s = sl * r$, since $sl = 1/d$ then $s = 1/d * r = (r/d)$.

□ **Information** such as the *number of index levels is easy* to maintain because it *does not change very* often. **However**, other information may **change frequently**;

- For example, the *number of records r* in a file changes *every time* a record is *inserted* or *deleted*.

□ The query optimizer will need reasonably close but not necessarily completely *up-to-the-minute values of these parameters* for use in estimating the *cost of various execution strategies*.



1.6) Overview of Query Optimization in ORACLE

ORACLE:

The **ORACLE DBMS** provides
two different approaches to
query optimization:

*The
Rule
Based
Approach*

*The
Cost
Based
Approach*



1.6) Overview of Query Optimization in ORACLE

- With the *rule-based approach*, the **optimizer** chooses *execution plans* based on *heuristically ranked operations*.
- ORACLE maintains *a table of ranked access paths*, where a *lower ranking implies* a **more efficient** approach.
 - ❑ The *access paths range* from table access by **ROWID** (*most efficient*) — where **ROWID** specifies the *record's physical address* that includes:
 1. The **data file**,
 2. **Data block**, and
 3. *Row offset*within the block— to a *full table scan* (least efficient)—*where all rows in the table are searched* by doing **multi-block** reads.
- However, the *rule-based approach* is *being phased out* in **favor** of the *cost-based approach*, where the *optimizer examines alternative access paths and operator algorithms* and chooses the execution plan with *lowest estimated cost*.



1.6) Overview of Query Optimization in ORACLE

- The *catalog tables* containing *statistical information* are used in a *similar fashion*, as described in Section 1.5.
- The **estimated query cost** is *proportional* to the *expected elapsed time* needed to *execute the query* with the *given execution plan*.
 - ❑ The **ORACLE optimizer calculates** this **cost** based on the *estimated usage of resources*, such as:
 - ✓ *Input/Output*,
 - ✓ *CPU time* and
 - ✓ *Memory Needed*
- The goal of *cost-based optimization* in **ORACLE** is to *minimize the elapsed time* to process the entire query.
- The *cost-based optimization* of ORACLE-8 is a good example of the *sophisticated approach* taken to *optimize SQL queries in commercial RDBMSs*.



1.7) Semantic Query Optimization

- A *different approach* to query optimization, called **semantic query optimization**, has been *suggested*.
- This **technique**, which may be used in *combination with the techniques discussed previously*, *uses constraints specified on the database schema* — such as:
 - ❑ *Unique attributes* and
 - ❑ *Other more complex constraints* — in order to *modify one query into another query* that is more efficient to execute.
- We will not discuss this approach in detail *but only illustrate it with a simple example*. Consider the *SQL query*:

```
SELECT  E.LNAME, M.LNAME
FROM    EMPLOYEE AS E, EMPLOYEE AS M
WHERE   E.SUPERSSN=M.SSN AND
          E.SALARY.M.SALARY
```



1.7) Semantic Query Optimization

- This query **retrieves** the names of *employees who earn more than their supervisors*.
 - Suppose that *we had a constraint on the database schema* that stated that **no employee can earn more than his or her direct supervisor**.
- If the semantic query *optimizer checks for the existence* of this **constraint**, it *need not execute the query* at all because it knows that the result of the **query will be empty**.
 - This may **save considerable time** if the **constraint checking can be done efficiently**.
 - **However**, searching through *many constraints to find those that are applicable to a given query* and that **may semantically optimize it can also be quite time-consuming**.
- With the *inclusion of active rules* in database systems *semantic query optimization techniques* may *eventually be fully incorporated* into the **DBMSs of the future**.

//End Chapter-1!



Question?
Suggestion!